



PERTEMUAN 06

Filter Spasial

Pengolahan Citra dan Visi Komputer · RTI235007



Overview



Konsep filter

Tujuan filter, domain frekuensi dan domain spasial



Konvolusi

Kernel, piksel tetangga, contoh hitung, masalah tepi



Jenis filter

Sharpening, blurring, emboss, deteksi tepi



01

Konsep filter

Apa itu filter dan di domain mana ia bekerja



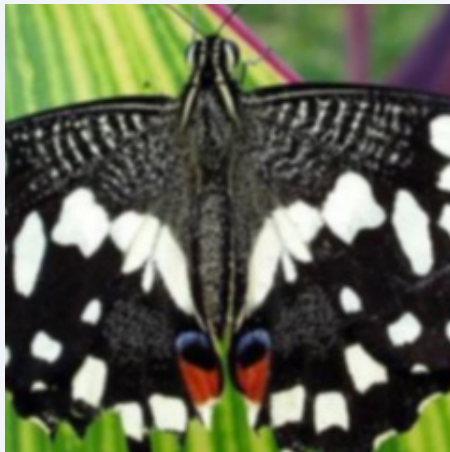


Apa itu proses filter?

Cara mengekstraksi bagian tertentu dari sekumpulan data dengan menghilangkan bagian yang tidak diinginkan.

Tujuan

Membuat citra tampak lebih baik, atau tampak lebih jelas untuk dianalisis.



Menekan frekuensi tinggi

Menghaluskan citra (smoothing). Detail halus dan derau berkurang.

Contoh: blurring, reduksi derau



Menekan frekuensi rendah

Memperjelas citra atau mendeteksi tepi. Perbedaan antarpiksel diperkuat.

Contoh: sharpening, deteksi tepi



Dua domain filter

Filter dapat bekerja pada spektrum frekuensi citra, atau langsung pada piksel-pikselnya.

Domain frekuensi

Citra diubah ke domain frekuensi, dikalikan dengan sebuah fungsi filter frekuensi, lalu hasilnya ditransformasikan kembali ke domain spasial.

Citra ➤ Transformasi ➤ × Filter ➤ Invers ➤ Citra baru

Contoh: transformasi Fourier

Domain spasial

Kumpulan piksel pada citra dimanipulasi secara langsung untuk menghasilkan citra baru.

Citra ➤ Konvolusi dengan kernel ➤ Citra baru

Fokus pertemuan ini

Filter domain spasial

Konvolusi citra input $f(i,j)$ dengan fungsi filter $h(i,j)$. Fungsi filter disimulasikan dalam bentuk kernel diskret tertentu.

$$g(i,j) = h(i,j) \odot f(i,j)$$



02

Konvolusi

Kernel, piksel tetangga, dan cara menghitungnya





Konvolusi: kernel yang bergeser di atas citra

1

Penjumlahan hasil kali

Setiap elemen kernel dikalikan dengan piksel di bawahnya, lalu semua hasilnya dijumlahkan.

2

Kernel bergeser

Kernel dioperasikan secara bergeser di atas citra input $f(x)$, piksel demi piksel.

3

Piksel tetangga

Nilai sebuah piksel dianggap dipengaruhi oleh piksel-piksel tetangganya (neighbouring pixels).

Siapa saja tetangganya?

Bentuk kernel menentukan piksel tetangga mana yang ikut menentukan nilai piksel P.

A	B	C
H	P	D
G	F	E

4-tetangga +

A	B	C
H	P	D
G	F	E

4-tetangga ×

A	B	C
H	P	D
G	F	E

8-tetangga

P = piksel yang sedang dihitung ■ = tetangga yang dipakai

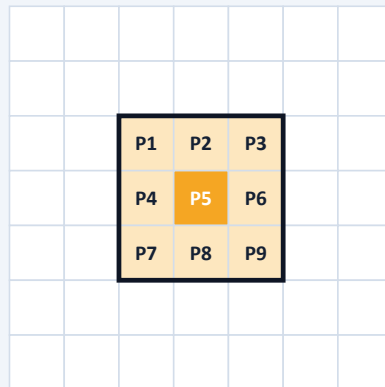


Rumus konvolusi

$$g(x, y) = f(x, y) \odot h(x, y) = \sum_{a=-\infty}^{\infty} \sum_{b=-\infty}^{\infty} f(a, b) h(x-a, y-b)$$

A	B	C
D	E	F
G	H	I

Kernel h



Citra f



$g(i, j)$

Output

Cara membacanya

f = citra input, h = kernel, $\odot$ = operator konvolusi.

Untuk kernel 3x3, penjumlahan cukup dilakukan pada 9 posisi di sekitar piksel.

$$\begin{aligned} g(i, j) = & A \cdot P1 + B \cdot P2 + C \cdot P3 \\ & + D \cdot P4 + E \cdot P5 + F \cdot P6 \\ & + G \cdot P7 + H \cdot P8 + I \cdot P9 \end{aligned}$$



Contoh 1: menghitung satu piksel

Kernel 3×3 dengan semua bobot $\frac{1}{4}$ diletakkan dengan pusatnya di piksel (2,2).

5	5	6	6
5	4	4	7
0	0	2	2
0	1	1	3

Citra input



$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$

Kernel

Perhitungan

$$\begin{aligned}g(2,2) &= 5 \times \frac{1}{4} + 5 \times \frac{1}{4} + 6 \times \frac{1}{4} \\&+ 5 \times \frac{1}{4} + 4 \times \frac{1}{4} + 4 \times \frac{1}{4} \\&+ 0 \times \frac{1}{4} + 0 \times \frac{1}{4} + 2 \times \frac{1}{4} \\&= 1,25 + 1,25 + 1,5 + 1,25 \\&+ 1 + 1 + 0 + 0 + 0,5\end{aligned}$$

$$g(2,2) = 7,75$$



Contoh 2: kernel 3×3 pada citra 5×5

$f(x, y)$

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

Kernel $h(x, y)$



0	-1	0
-1	*4	-1
0	-1	0

Catatan: tanda * menunjukkan posisi (0,0) dari kernel, yaitu titik yang diletakkan tepat di atas piksel yang sedang dihitung.

Langkah menghitung

- 1 Letakkan pusat kernel (*) di atas piksel (1,1).
- 2 Kalikan setiap elemen kernel dengan piksel di bawahnya.
- 3 Jumlahkan, lalu tulis hasilnya di posisi yang sama pada citra output.
- 4 Geser kernel satu piksel, ulangi sampai seluruh citra.



Langkah demi langkah: baris pertama

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4



	3			

Posisi (1,1)

Hasil = 3

$$(0 \times 4) + (-1 \times 4) + (0 \times 3) + (-1 \times 6) + (4 \times 6) + (-1 \times 5) + (0 \times 5) + (-1 \times 6) + (0 \times 6) = 3$$

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4



	3	0		

Posisi (1,2)

Hasil = 0

$$(0 \times 4) + (-1 \times 3) + (0 \times 5) + (-1 \times 6) + (4 \times 5) + (-1 \times 5) + (0 \times 6) + (-1 \times 6) + (0 \times 6) = 0$$

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4



	3	0	2	

Posisi (1,3)

Hasil = 2

$$(0 \times 3) + (-1 \times 5) + (0 \times 4) + (-1 \times 5) + (4 \times 5) + (-1 \times 2) + (0 \times 6) + (-1 \times 6) + (0 \times 2) = 2$$

Trik: suku yang dikali 0 boleh dilewati. Cukup hitung $4 \times$ pusat – (atas + bawah + kiri + kanan).



Langkah demi langkah: baris kedua

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

	3	0	2	
	0			

Posisi (2,1)

Hasil = 0

$$(0 \times 6) + (-1 \times 6) + (0 \times 5) + (-1 \times 5) + (4 \times 6) + (-1 \times 6) + (0 \times 6) + (-1 \times 7) + (0 \times 5) = 0$$

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

	3	0	2	
	0	2		

Posisi (2,2)

Hasil = 2

$$(0 \times 6) + (-1 \times 5) + (0 \times 5) + (-1 \times 6) + (4 \times 6) + (-1 \times 6) + (0 \times 7) + (-1 \times 5) + (0 \times 5) = 2$$

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

	3	0	2	
	0	2	6	

Posisi (2,3)

Hasil = 6

$$(0 \times 5) + (-1 \times 5) + (0 \times 2) + (-1 \times 6) + (4 \times 6) + (-1 \times 2) + (0 \times 5) + (-1 \times 5) + (0 \times 3) = 6$$

Latihan: hitung sendiri baris ketiga, posisi (3,1), (3,2), dan (3,3). Cocokkan dengan hasil di slide berikutnya.



Hasil akhir, dan masalah di tepi citra

Hasil konvolusi

	3	0	2	
	0	2	6	
	6	0	2	

Hanya 3×3 piksel yang terisi. Baris dan kolom pinggir masih kosong.

Permasalahan

4	4	3	5	4	?
6	6	5	5	2	?
5	6	6	6	2	?
6	7	5	5	3	
3	5	2	4	4	

Saat pusat kernel berada di piksel pinggir, sebagian kernel keluar dari citra. Tidak ada nilai piksel ("?",) untuk dikalikan.

Pertanyaan

Citra 5×5 dengan kernel 3×3 hanya menghasilkan 3×3 nilai. Bagaimana cara mengisi piksel pinggir agar ukuran output sama dengan input?



Tiga solusi untuk piksel pinggir

1 Abaikan piksel pinggir

4	4	3	5	4
6	3	0	2	2
5	0	2	6	2
6	6	0	2	3
3	5	2	4	4

Piksel pinggir tidak dikonvolusi, nilainya dibiarkan seperti semula.

Paling sederhana, tepi tidak terfilter

2 Duplikasi elemen citra

5	5	5	6	6	6
5	5	5	6	6	6
5	5	4	4	7	7
0	0	0	2	2	2
0	0	1	1	3	3
0	0	1	1	3	3

Baris dan kolom pinggir disalin ke luar citra sebelum konvolusi.

OpenCV: `cv2.BORDER_REPLICATE`

3 Isi dengan 0 / konstanta

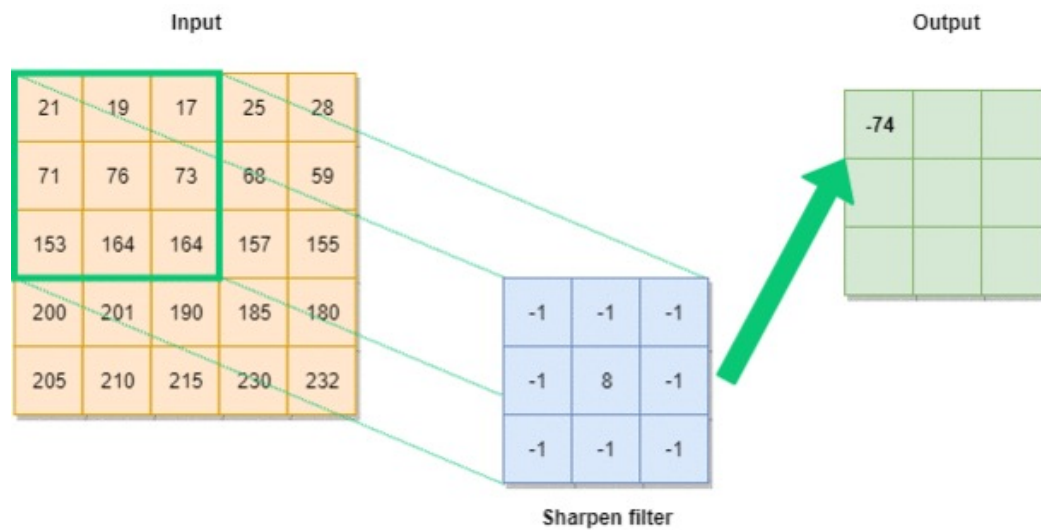
0	0	0	0	0	0
0	5	5	6	6	0
0	5	4	4	7	0
0	0	0	2	2	0
0	0	1	1	3	0
0	0	0	0	0	0

Elemen "?" di luar citra dianggap bernilai 0 atau konstanta lain.

OpenCV: `cv2.BORDER_CONSTANT`



Operasi yang sama di balik CNN



AIGeekProgrammer.com © 2019

Sumber gambar: AIGeekProgrammer.com © 2019

Coba hitung ulang

Jendela 3×3 kiri atas dikalikan dengan sharpen filter (pusat 8, tetangga -1):

$$\begin{aligned} 8 \times 76 &= (21+19+17 \\ &+ 71+73+153 \\ &+ 164+164) \\ &= 608 - 682 = -74 \end{aligned}$$

Pada Convolutional Neural Network (CNN), nilai kernel tidak ditentukan manual, melainkan dipelajari dari data.



Algoritma konvolusi

Pseudocode

```
For x = 0 to image_width-1
  For y = 0 to image_height-1
    z(x,y) = 0
    For k1 = 0 to filter_width-1
      For k2 = 0 to filter_height-1
        z(x,y) = z(x,y) + H(k1,k2)
          * Img_pixel(x+k1, y+k2)
      next k2
    next k1
  next y
next x
```

Di Python (OpenCV)

Empat loop bersarang di kiri cukup ditulis dalam satu baris:

```
kernel = np.array([[ 0,-1, 0],
                  [-1, 4,-1],
                  [ 0,-1, 0]],
                  dtype=np.float32)
hasil = cv2.filter2D(img, -1, kernel)
```

Catatan: cv2.filter2D tidak membalik kernel (korelasi). Untuk kernel simetris seperti contoh ini hasilnya sama dengan konvolusi.



03

Jenis filter

Sharpening, blurring, emboss, dan deteksi tepi





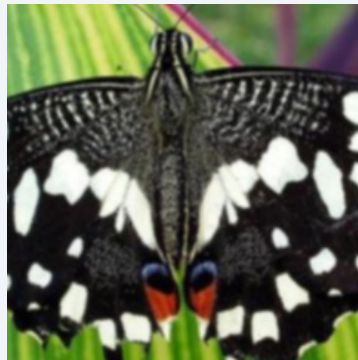
Empat jenis filter spasial

Semuanya konvolusi. Yang membedakan hanyalah isi kernelnya.



1 Sharpening *penajaman*

Menonjolkan detail dan tepi.



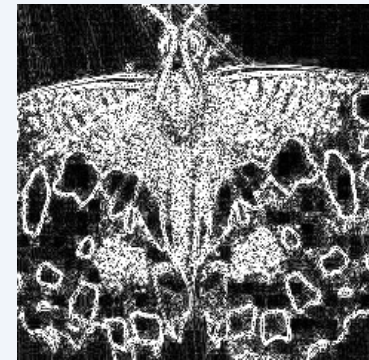
2 Blurring *pengaburan*

Menghaluskan citra,
meredam derau.



3 Emboss *efek timbul*

Menonjolkan tepi pada satu
arah.



4 Edge detection *deteksi tepi*

Menyisakan batas objek saja.

Potongan butterfly.jpg (OpenCV sample data) yang difilter dengan kernel pada slide-slide berikutnya.



Sharpening (penajaman)

Intensitas piksel pusat diperkuat, berlawanan arah terhadap piksel-piksel tetangganya.

0	-1	0
-1	5	-1
0	-1	0

Kernel A

Sumber: docs.gimp.org

1	1	1
1	-8	1
1	1	1

Kernel B

Sumber: imagesincontext.com



Asli



Kernel A

Jumlah bobot A = 1 sehingga kecerahan terjaga. **Jumlah bobot B = 0** sehingga hasilnya peta tepi (Laplacian); citra tajam = citra – hasil B.



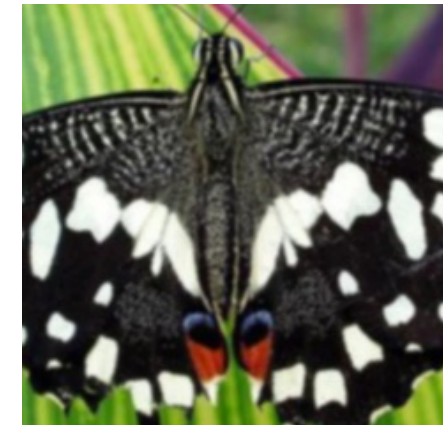
Blurring (pengaburan)

Nilai piksel pusat dibuat mendekati piksel tetangganya, sehingga perbedaannya berkurang.

1/256	1	4	6	4	1
	4	16	24	16	4
	6	24	36	24	6
	4	16	24	16	4
	1	4	6	4	1
Gaussian 5×5					
1/16					
Gaussian 3×3					



Asli



Gaussian 5×5

Dibagi 256 (atau 16) agar jumlah bobot = 1: citra menjadi halus tanpa berubah kecerahannya. Bobot terbesar di pusat, makin kecil ke tepi.



Emboss

Memperkuat tepi pada satu arah tertentu, tanpa menghilangkan warna lainnya.

-2	-1	0
-1	1	1
0	1	2

Kernel A

Sumber: docs.gimp.org

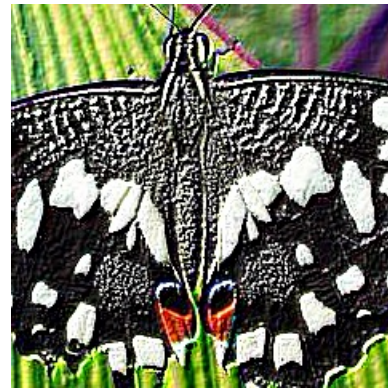
1	0	0
0	0	0
0	0	-1

Kernel B (+128)

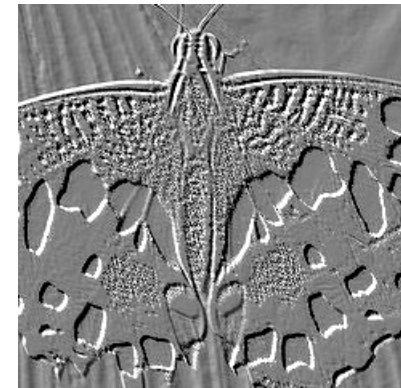
Sumber: imagesincontext.com



Asli



Kernel A



Kernel B

Kernel A berjumlah 1, warna tetap terlihat. **Kernel B** berjumlah 0, sehingga hasilnya ditambah 128 agar area rata tampak abu-abu.



DATA 1

DATA 1

DATA 1

Terima kasih!

Team Teaching PCVK

